EnergyWatch AI

INCIDENT

Incident Report INC-0001

Post-mortem of the LLM provider outage drill: timeline, runbook steps applied, evidence and improvements.

Projet	**EnergyWatch AI, plateforme multi-agents (LangGraph)**
Secteur	**Énergie**
Cadre	**Mini-projet S5 · Agent en production**
Version	**1.2.0**
Date	**30 octobre 2026**
Dépôt	**github.com/bouincha/Energy-Watch-AI**

Incident Report INC-0001: LLM provider outage (game-day drill)

Field	Value
Incident	INC-0001
Runbook	RB-001: LLM provider outage
Severity	SEV-2
Type	Planned drill (chaos injection), run against the production Docker image
Date	2026-10-30
Detected	11:16:00.918 UTC (automatically, by the monitoring agent)
Resolved	11:16:32.064 UTC (automatically, by the monitoring agent)
MTTR	**31.15 s**
Affected runs	`drill-outage-0001` (plus `drill-outage-0002`, served in degraded mode while the breaker was open)
Data loss	None
User impact	Answers stayed available, flagged as *degraded*: no AI-written narrative, and every number was still correct

Summary

To check that runbook RB-001 really works, we injected a total LLM provider outage with the chaos controller (`POST /api/chaos {"llm_outage": true}`) on the containerized application (`energywatch-ai:local`, `ADMIN_API_KEY` set). The circuit breaker opened after 3 consecutive failures, the monitoring agent opened INC-0001, the agents switched to their deterministic fallbacks, and the incident resolved itself after the outage ended and the breaker's cool-down elapsed.

Timeline (UTC, from the incident record and the container logs)

Time	Event	Correlation ID
11:15:59	Operator enables `llm_outage` (audited as `chaos.updated`)	-
11:15:59	Run "Give me the full monthly report" starts; LLM calls fail (`ChaosError`)	`drill-outage-0001`
11:16:00.918	3rd consecutive failure: breaker `closed → open`	`drill-outage-0001`
11:16:00.918	**INC-0001 opened** (Detect) and degraded mode applied (Mitigate)	`drill-outage-0001`

Time	Event	Correlation ID
11:16:00	Run finishes `degraded` in 1,871 ms: 3 LLM calls, 3 failed, answer delivered	`drill-outage-0001`
11:16:01	Second run finishes `degraded` in **54 ms**: 0 LLM calls (short-circuited by the open breaker)	`drill-outage-0002`
11:16:01	`/ready` reports `status=degraded`, `circuit_breaker=open`, `open_incidents=[INC-0001]`	-
11:16:01	Operator disables `llm_outage` (end of the simulated outage)	-
11:16:31	Cool-down (30 s) elapses: breaker becomes `half_open`	-
11:16:32.064	Probe call succeeds: breaker `half_open → closed`, **INC-0001 resolved**	`drill-recovery-0001`

Log extract (`docker logs ewa-test | grep -E "circuit breaker|incident"`):


```
{"ts": "2026-10-30T11:16:00.918Z", "level": "WARNING", "logger": "energywatch.resilience",
"msg": "circuit breaker 'llm' closed → open", "correlation_id": "drill-outage-0001", "breaker":
"llm", "old_state": "closed", "new_state": "open"}
{"ts": "2026-10-30T11:16:00.918Z", "level": "ERROR", "logger": "energywatch.incidents", "msg":
"incident INC-0001 opened: LLM provider outage", "correlation_id": "drill-outage-0001",
"incident_id": "INC-0001", "runbook": "RB-001", "severity": "SEV-2"}
{"ts": "2026-10-30T11:16:32.064Z", "level": "INFO", "logger": "energywatch.resilience", "msg":
"circuit breaker 'llm' half_open → closed", "correlation_id": "drill-recovery-0001", "breaker":
"llm", "old_state": "half_open", "new_state": "closed"}
{"ts": "2026-10-30T11:16:32.064Z", "level": "WARNING", "logger": "energywatch.incidents", "msg":
"incident INC-0001 resolved", "correlation_id": "drill-recovery-0001", "incident_id": "INC-
0001", "mttr_s": 31.15, "resolved_by": "monitoring-agent"}
```


Runbook steps applied

RB-001 step	Applied?	Evidence
1. Detect	✅ automatic	Incident timeline entry "Detect", log `incident INC-0001 opened`
2. Mitigate (degraded mode)	✅ automatic	All LLM agents' spans `degraded`; answer contains the degraded banner; 5 anomalies still reported
3. Communicate	✅ automatic	`/ready` = degraded; sidebar shows `LLM: … · open`; incident visible on Monitoring
4. Diagnose	✅	Logs show `error_type=chaos` for each failed call, which confirms the root cause
5. Recover (half-open probe)	✅ automatic	Timeline entry "Recover: half-open probe succeeded"
6. Resolve	✅ automatic	Timeline entry "Resolve", MTTR 31.15 s

What went well

- **No user-facing outage:** both runs returned complete, correct answers with every number, because figures come from deterministic tools.
- **Fail fast:** once the breaker opened, runs took 54 ms instead of waiting on a dead provider.
- **Traceability:** a single `grep` on the correlation ID reconstructs the whole incident.

What could be improved

1. **First failing run latency (1.9 s):** each failing call is retried before the breaker opens. *Action:* keep `LLM_MAX_RETRIES=2` (good for transient errors), but make the backoff configurable for providers that fail hard.
2. **The MTTR is dominated by the 30 s cool-down.** That's acceptable for a SEV-2. A shorter cool-down would make more probe calls against a failing provider.
3. **Telemetry lives in memory:** incidents are lost on restart, although the JSON logs are kept. *Action (v1.1):* export the metrics and incidents to an external store.

How to reproduce

```
curl -X POST $URL/api/chaos -H "X-API-Key: $ADMIN_API_KEY" -H "Content-Type: application/json" -
d '{"llm_outage": true}'
curl -X POST $URL/api/agent/ask -H "Content-Type: application/json" -d '{"query": "Give me the
full monthly report"}'
curl $URL/api/incidents
curl -X POST $URL/api/chaos -H "X-API-Key: $ADMIN_API_KEY" -H "Content-Type: application/json" -
d '{"llm_outage": false}'
sleep 31 && curl -X POST $URL/api/agent/ask -H "Content-Type: application/json" -d '{"query":
"Any anomalies at the plant?"}'
curl $URL/api/incidents   # INC-0001 is now resolved, with its MTTR
```

The same scenario is covered automatically by `tests/test_incidents.py::test_llm_outage_incident_lifecycle`.